C 8205 – Bedienungsanleitung für den Python-Interpreter

Testprogramme für den Cellatron C 8205 schreiben, ausführen und die Ergebnisse lesen

Gilt für: `c8205_interp.py`, Fassung vom 04.10.2026 (81 299 Byte, SHA-256 beginnt mit `e715ccaef1d7`), zusammen mit dem Wrapper `c8205_interpreter.sh` bzw. `.bat`. Alle Beispiele dieser Anleitung wurden mit dieser Fassung ausgeführt; die Ausgaben sind übernommen, nicht von Hand geschrieben.

Zielgruppe: Personen, die Testprogramme für den C 8205 schreiben und durchrechnen, ohne den Aufbau des Interpreters zu kennen. Kenntnisse über Oktalzahlen und Register werden nicht vorausgesetzt; Begriffe stehen im Glossar (Anhang A).

Quelle: C8205-Interpreter-Python_Interpreter.md (Abschnitte 1–13, Stand 04.10.2026) und die Testdefinition aus C8205-Interpreter-Durchführung_eines_Tests.md. Diese Anleitung erklärt die Bedienung des Interpreters, nicht den Rechner; zum Rechner siehe die PDFs „Rechnerbeschreibung“ und „Bedienpult und Peripherie“.

Legende

`$ befehl`	Eingabe im Terminal (Befehl, den Sie tippen).
Normale Zeile	Ausgabe des Interpreters.
Dunkelgelb	Ausgabe des **Rechners** über das Schreibwerk (Kanal K2). Im Schwarz-Weiß-Druck erkennen Sie sie an der Zeile selbst (z. B. „Jahr 1972“) und am Hinweis im Text.
Rot	Fehlermeldung. Jede Meldung beginnt mit **„FEHLER:“**, damit sie auch ohne Farbe auffällt.
Hinweis / Achtung	Blauer Balken: nützlich. Orangefarbener Balken: hier scheitern Tests häufig.

Inhaltsverzeichnis

1 Zu dieser Anleitung . . . 3
1.1 Was der Interpreter tut . . . 3
1.2 Wie die Anleitung aufgebaut ist . . . 3
1.3 Schreibweisen . . . 3
1.4 Stand und Fassung . . . 3

2 Erste Schritte . . . 4
2.1 Voraussetzungen . . . 4
2.2 Ordner . . . 4
2.3 Der erste Lauf . . . 4

3 Tests schreiben . . . 6
3.1 Trommeldatei . . . 6
3.2 Testmarken . . . 6
3.3 Tastenfolge . . . 6
3.4 Prüfungen (optional) . . . 8
3.5 Formale Prüfung vor dem Start . . . 8
3.6 Geräte in der Testdatei . . . 8

4 Interpreter starten . . . 10
4.1 Kommandozeile . . . 10
4.2 Mehrere Tests, Protokollort . . . 11
4.3 Wrapper . . . 11
4.4 Rückkehrcodes . . . 11

5 Ein- und Ausgabe . . . 12
5.1 Schreibwerk-Ausgabe (K2) . . . 12
5.2 Lochstreifenleser (K1, K2) . . . 12
5.3 Lochstreifenstanzer (K1) . . . 13
5.4 Pult-Tastatur und Schreibwerk-Eingabe (K0) . . . 13

6 Ergebnisse lesen . . . 14
6.1 Bildschirmausgabe . . . 14
6.2 Die Endzeile . . . 14
6.3 Diagnose bei Befehl 164 . . . 14
6.4 Knotenprotokoll und Protokolldatei . . . 15
6.5 Ergebnisprüfung . . . 16

7 Wenn etwas schiefgeht . . . 17
Beispiele . . . 18

8 Beispiele Schritt für Schritt . . . 20
8.1 Ausgabe auf dem Schreibwerk . . . 20
8.2 Alle Bedientasten in einer Tastenfolge . . . 20
8.3 Zeichen vom Lochstreifen lesen und auf den Stanzer ausgeben . . . 21
8.4 Diagnose, Speicherauszug und Prüfungen . . . 22

9 Kurzreferenz . . . 24

Anhang A: Glossar . . . 25

Anhang B: Grenzen des Interpreters . . . 25

Anhang C: Verweise auf das Regelwerk . . . 26

1 Zu dieser Anleitung

1.1 Was der Interpreter tut

Der Interpreter führt Programme des Rechners C 8205 interpretativ aus. Er läuft den Knotengraphen des Befehlsablaufs (Phasen I bis IV) ab und bildet Register, Flags, Trommelspeicher und Peripherie nach. Alle Zeiten (Wortzeit, Wartezeit auf die Trommel) sind 0; die Trommeldrehung wird nicht nachgebildet.

Sie benutzen ihn, um

- Testprogramme des C 8205 nachvollziehbar durchzurechnen (auf Wunsch Knoten für Knoten protokolliert),
- das Verhalten des Regelwerks zu prüfen,
- Ein- und Ausgabe (Schreibwerk, Lochstreifenleser, Stanzer, Pult-Tastatur) am Terminal zu erproben.

> **Hinweis** **Grundsatz „laut scheitern“:** Der Interpreter läuft den Graphen wortlautgetreu ab. Ist etwas unklar oder fehlt eine Festlegung, meldet er das und hält an; er rät nie und „repariert“ nichts stillschweigend. Ein Test, der nicht startet, ist daher kein Defekt, sondern eine Meldung, die Sie lesen sollten.

1.2 Wie die Anleitung aufgebaut ist

Kapitel	Inhalt	Wann lesen?
2 Erste Schritte	Voraussetzungen, Ordner, erster Lauf	beim ersten Mal
3 Tests schreiben	Testdatei, Marken, Tastenfolge, Prüfungen, Geräte-Zeilen	beim Anlegen eines Tests
4 Interpreter starten	Aufruf, Schalter, Wrapper, Rückkehrcodes	bei jedem Lauf
5 Ein- und Ausgabe	Kanäle, Schreibwerk, Leser, Stanzer, Tastatur	bei Zeichen-Ein-/Ausgabe
6 Ergebnisse lesen	Endzeile, Diagnose, Protokoll, Ergebnisprüfung	nach jedem Lauf
7 Wenn etwas schiefgeht	Meldungen mit Ursache, Abhilfe, Rückkehrcode	bei Meldungen
8 Beispiele	Beispiele Schritt für Schritt zum Nachmachen	zum Ausprobieren
9 Kurzreferenz	eine Seite zum Ausdrucken	zum Nachschlagen

1.3 Schreibweisen

- Dateinamen, Schalter und Eingaben stehen in Festbreitenschrift.
- Zahlen in Testdateien und Registerwerten sind **oktal** (Ziffern 0–7). Wo Ziffern in der Anleitung dezimal gemeint sind, steht es dabei.
- Kanäle heißen immer K0 bis K3; Geräte heißen Lochstreifenleser (Kürzel Leser), Lochstreifenstanzer (Stanzer), Schreibwerk (SW).
- Begriffe: **Testdatei** (die Datei, die Sie dem Interpreter geben), **Trommeldatei** (ihr erster Teil: der Speicherinhalt), **Tastenfolge** (ihr zweiter Teil: die Bedienhandlungen).

1.4 Stand und Fassung

Die Anleitung gilt für die auf der Titelseite genannte Fassung. Ändert sich der Interpreter (neuer Schalter, neues Token, neue Meldung), muss die Anleitung neu erzeugt werden; Abweichungen zwischen Anleitung und Interpreter sind dann ein Fehler der Anleitung.

Neu gegenüber der Anleitung vom 03.10.2026 (Fassung 4bb5c05c2a8a): `--pruefe-stanzer1`, `--pruefe-sw-ausgabe`, Geräte-Zeilen auch für Leser und Stanzer, Kanaltasten K0 bis K3, Rückkehrcodes 0, 2, 4, 6, 8, 16, „Irrung Zeichen“ am Leser, Zelleninhalt beim Eingabe-Lesebefehl, Stub bei Ausgabe-Halt ohne Gerät, Überlauftest nach Ausgabe-Halt und Anwahl sowie die geänderte Befehlsentnahme nach dem X-Abbruch (Abschnitt 8.4).

> **Hinweis** **Stand der Beispiele.** Alle Beispiele wurden mit der Fassung dieser Anleitung ausgeführt (Interpreter und Regelwerk-Dateien vom 04.10.2026); die Ausgaben sind aus den Läufen übernommen. Die Zahl hinter „Knoten:“ ist **fassungsabhängig** und kein Sollwert; sie ändert sich, wenn der Knotengraph geändert wird. Ordnernamen und Pfade der Beispiele sind frei gewählt.

2 Erste Schritte

2.1 Voraussetzungen

- **Python 3.** Es wird nur die Standardbibliothek benutzt; es ist nichts zu installieren.
- **Verzeichnis** `markdown-dateien/` **neben** `c8205_interp.py` (Vorgabe für `--mdverz`) mit `C8205-Interpreter-Eingänge_der_Knoten.md` (Knotengraph) und `C8205-Interpreter-Beschreibung_der_Knoten.md` (Variablentabelle T2). Der Interpreter liest beide bei jedem Start; fehlt eine, endet der Aufruf mit Rückkehrcode 2.
- Optional der Wrapper `c8205_interpreter.sh` (Linux/macOS) bzw. `c8205_interpreter.bat` (Windows), ebenfalls im Ordner des Skripts.
- Ein Terminal mit ANSI-Farben (für Dunkelgelb und Rot). Ohne Terminal oder mit gesetztem `NO_COLOR` erscheint reiner Text.

Hinweis Der Dateiname der Knotengraph-Datei enthält ein „ä“. Findet der Interpreter sie nicht (z. B. wegen der Zeichenkodierung eines Archivs), geben Sie das Verzeichnis mit `--mdverz` an.

2.2 Ordner

```
Projektordner/
  c8205_interp.py
  c8205_interpreter.sh   c8205_interpreter.bat
  markdown-dateien/      Regelwerk (u. a. Eingänge_der_Knoten.md, Beschreibung_der_Knoten.md)
  Tests/                 Testdateien (.txt)
  Test-Protokolle/       Protokolle (.log), vom Wrapper angelegt
  virtuelle_Geraete/     Ablage für virtuelle Geräte (NICHT automatisch benutzt)
```

Hinweis Die Ordner unter `virtuelle_Geraete/` (Leser1/, Leser2/, Stanzer1/, SW-Ausgabe/, T-SW-Eingabe/) werden **nicht automatisch** benutzt. Sie wirken nur, wenn Sie den Schalter beim Aufruf angeben – auch beim Wrapper: `./c8205_interpreter.sh Tests/x.txt --leser1 virtuelle_Geraete/Leser1` (Abschnitt 4.3) – oder wenn die Testdatei den Pfad als Geräte-Zeile selbst mitbringt (Abschnitt 3.6).

2.3 Der erste Lauf

Legen Sie die Datei `Tests/0015-030.txt` mit folgendem Inhalt an. Das Programm wählt K2 an und gibt auf dem Schreibwerk einen Text aus.

```
Trommeldatei:
#DOK-0015 Test Ausgabebefehl
#PROT STUFE=2
0001: 00007240000
7240: 00004500030
7241: 00002720030
7242: 00000570030
7243: 00001210030
7244: 00001170030
7245: 00000610030
7246: 00000700030
7247: 00001110030
7250: 00000320030
7251: 00000010030
7252: 00000310030
7253: 00000070030
7254: 00000020030
7255: 00000015164
7256: 00000000020

Tastenfolge:
H G M-drücken E M-loslassen R
```

Starten Sie den Test mit dem Wrapper:

```
$ ./c8205_interpreter.sh Tests/0015-030.txt
=== Test 0015-030 (#PROT STUFE=2) ===

Jahr 1972
-- Diagnose --
#DOK-0015 Test Ausgabebefehl
Ende: Unbedingter Stop (US:=1) | Knoten: 308 | BZ=7257 AC=00000002000 AR=0000 | FE=0 FUE=0 BS=0 US=1 E=0 A=0
  V1=0 V2=0
Protokolldatei: Test-Protokolle/0015-030.log
```

Das Programm hat den Text „Jahr 1972“ ausgegeben (die Leerzeile davor ist ein Zeilenumbruch des Programms) und mit einem unbedingten Stopp gehalten. Die Zeile „Ende: …“ fasst den Endzustand zusammen (Kapitel 6). Das ausführliche Protokoll liegt in `Test-Protokolle/0015-030.log`.

> **Achtung** Die Schreibweise der Buchstaben („Jahr“, „JAHR“ oder „jahr“) hängt von der Zeichenlage KB/GB des Schreibwerks ab, die beim Start **zufällig** gesetzt wird. Dieses Programm gibt selbst Umschaltzeichen aus; ohne solche Zeichen wäre die Groß-/Kleinschreibung zufällig. Auch die „Knoten“-Zahl der Endzeile ist **fassungsabhängig** und kein fester Sollwert.

3 Tests schreiben

Eine Testdatei besteht aus zwei Teilen (optional drei): Der Teil `Trommeldatei:` legt den Anfangsinhalt des Speichers fest, der Teil `Tastenfolge:` die Bedienhandlungen am Pult vor dem Start, der Teil `Prüfungen:` (optional) die erwarteten Ergebnisse.

3.1 Trommeldatei

Jede Zeile hat die Form `Adresse: Wert [# Bemerkung]`. Die Adresse hat **vier** Oktalstellen, der Wert **elf**. Der Wert besteht aus vier Stellen „beliebig“ und sieben Stellen Befehl (vier Adresse + drei Operation): `00004500030` ist also Adresse 4500, Operation 030. Nicht genannte Zellen sind 0. Leerzeilen und Zeilen, die mit `#` beginnen, werden übergangen; ein Kommentar darf hinter einer Adresse:Wert-Zeile stehen, wenn mindestens ein Leerzeichen davorsteht.

```
# Beginn Programmbefehle
0000: 00003000211
0001: 00003001210
0002: 00000000020
# Beginn Datenbereich
3000: 00000000100 # Startwert
3001: 00000000045
```

Achtung Eine nicht genannte Zelle ist 0, und der Wert 0 ist als Befehl der **unbedingte Sprung nach Adresse 0000**. Läuft Ihr Programm über sein Ende hinaus, springt es zurück an den Anfang. Schließen Sie jedes Programm mit einem Stopp-Befehl `00000000020` ab.

Hinweis Wenn Sie in Assemblersprache programmieren, erzeugt der Assembler `c82ass` diese Zeilen für Sie (Datei `<name>.test`); Sie ergänzen nur Testmarken und Tastenfolge. Siehe PDF „Testhandbuch“, Kapitel 3.

3.2 Testmarken

Testmarken sehen aus wie Kommentare (`#`), haben aber eine Wirkung. Sie gehören **nur zum Test**; es gibt sie in der Hardware nicht.

Marke	Wirkung
`#DOK-xxxx Text`	Kommentar zum Test (xxxx oktal). Beim Befehl **164** („Leerbefehl“) mit der Adresse xxxx wird der Text als Diagnose ausgegeben, unabhängig von der Protokollstufe. Mehrfach zulässig.
`#SPEI-xxxx ssss-eeee`	Speicherbereich ssss bis eeee (oktal, ssss ≤ eeee), der beim Befehl 164 mit der Adresse xxxx ausgegeben wird. Unabhängig von `#DOK`: auch einzeln und mehrfach zulässig.
`#SPUR-xxxx aaaa`	Ganze Spur (Bahn), in der aaaa liegt: alle 32 Zellen in physischer Zugriffsreihenfolge der Trommel, ausgegeben beim Befehl 164 mit der Adresse xxxx. Unabhängig von `#DOK`.
`#PROT STUFE=n`	Protokollstufe des Tests (höchstens einmal): 0 = kein Knotenprotokoll, 1 = Knotenzeilen auf dem Bildschirm, 2 = Knotenzeilen nur in der Protokolldatei. Fehlt die Marke, gilt 1.

Achtung Die Diagnose erscheint nur, wenn der Test den Befehl **164** mit passender Adresse ausführt. Schreiben Sie ihn an die Stelle, an der Sie auswerten wollen: `00000020164` an Adresse 0002 löst die Marken mit xxxx = 0020 aus. Der Leerbefehl wird zuerst ausgeführt, die Diagnose erscheint danach. Eine automatische Zustandszeile gibt es nicht.

3.3 Tastenfolge

Die Tastenfolge (hinter `Tastenfolge:`) legt den Ausgangszustand fest. Die Token stehen durch Leerzeichen getrennt in einer oder mehreren Zeilen; Groß-/Kleinschreibung ist gleichgültig. Sie endet mit **R** (dann ist H am Ende gelöst) oder **im Halt** (H bleibt gesetzt, z. B. `H G Ü E`). Beides ist gültig. Endet sie weder mit R noch im Halt, startet der Test nicht.

Token	Wirkung
`H`	Schalter H-R auf „H“: Flag H := 1
`G`	Generallöschen: alle Flags außer H, T, BS, SP := 0, danach US := 1; AR, OR, BZ := 0; Kanal K0 angewählt (T := 1, bei gedrücktem SW: SW := 1)
`M+`, `M-drücken`, `M-gedrückt-halten`	M gedrückt halten: OR := OR ODER 025

Token	Wirkung
`M-`, `M-loslassen`	M loslassen: der Beitrag 025 im OR entfällt
`M`	Antippen = M+ und sofort M-: OR := OR ODER 025, BR bleibt aufgestellt; R startet bei Knoten 19
`E` (mit gehaltenem M)	BZ := BZ + 1, AR := BZ, AC := BZ+1 im Adressfeld; Befehlsentnahme gesperrt
`E` (ohne M)	nur bei H: Einzelschritt über den Knotengraphen ab Knoten 19 (Phase IV, I, II, III), Halt vor erneutem Eintritt in Knoten 19
`BS=1`, `SP=1` (auch `BS+`, `SP+`)	Schalter gedrückt und eingerastet: Flag BS bzw. SP := 1
`BS=0`, `SP=0` (auch `BS-`, `SP-`)	Schalter gelöst: Flag := 0. G löst die Schalter nicht.
`ST`	nicht an H gebunden, nur bei US oder BS wirksam (sonst keine Wirkung): OR := OR ODER 240, AR bleibt, US := 0; BR aufgestellt. Nach einem Stopp-Befehl ist das die Operation 26Z, nach G die Operation 240.
`Ü` (auch `UE`)	nur bei H: AR := 7777, OR := OR ODER 240; BR aufgestellt
`SW=1`, `SW+`, `SW-drücken`	Schalter SW gedrückt: Flag SW := 1, T := 0
`SW=0`, `SW-`, `SW-loslassen`	Schalter SW gelöst: Flag SW := 0, T := 1
`K0`, `K1`, `K2`, `K3`	Kanaltaste (rastend): Flag des gewählten Kanals := 1, die der anderen := 0; bei K0 zusätzlich T oder SW je nach Schalter SW. Ein im Programm ausgeführter Anwahlbefehl übersteuert die Einstellung.
`R`	Schalter H-R auf „R“: H := 0, US := 0; Ablauf beginnt bei Konnektor B (bei aufgestelltem BR bei Knoten 19)

Achtung `SW=1` muss **vor** `G` stehen, weil G daraus für K0 T oder SW ableitet. Die Kanaltasten `K0` … `K3` stehen **nach** `G`, weil G den Kanal K0 wieder anwählt. Nach `Ü`, `ST` oder `E` (ohne M) ist BR aufgestellt: folgt `R`, beginnt der Ablauf bei Knoten 19 statt bei Konnektor B.

Ungültig (der Test startet nicht): `M-` ohne `M+`, `M+` bei gedrücktem M, `R` oder Ende bei gedrücktem M, unbekannte Token. Die Eingabetasten (0…9, `+` …) gehören nicht in die Tastenfolge; Eingaben an K0 stehen in Abschnitt 5.4.

Muster: Sprung nach Speicherplatz n

Mit `H G M-drücken E … E M-loslassen R` (n-mal E) beginnt der Ablauf bei Adresse n. Beispiel n = 4 – das Programm steht ab Adresse 0004:

```
Trommeldatei:
#PROT STUFE=0
0004: 00000000020
Tastenfolge:
H G M+ E E E E M- R
```

```
$ python3 c8205_interp.py Tests/sprung-n.txt ; echo "Rückkehrcode: $?"
=== Test sprung-n (#PROT STUFE=0) ===
Ende: Unbedingter Stop (US:=1) | Knoten: 17 | BZ=0005 AC=00000005000 AR=0000 | FE=0 FUE=0 BS=0 US=1 E=0 A=0
  V1=0 V2=0
Rückkehrcode: 4
```

Der Ablauf hat bei Adresse 0004 begonnen und dort gehalten (BZ=0005). Der AC enthält – wie bei der Taste M üblich – im Adressfeld die um 1 erhöhte Adresse (…05000). Rückkehrcode 4: der Test enthält keine Prüfung (Abschnitt 4.4).

Tastenfolge im Halt

Endet die Tastenfolge ohne `R`, startet der Rechner nicht; der Lauf endet sofort mit dem Grund „Halt (Schalter H)“ und dem Rückkehrcode 4 (keine Prüfung angegeben, Abschnitt 4.3). Beispiel: Tastenfolge `H G`.

```
$ python3 c8205_interp.py Tests/halt-ende.txt ; echo "Rückkehrcode: $?"
=== Test halt-ende (#PROT STUFE=0) ===
Ende: Halt (Schalter H): Tastenfolge endet ohne R | Knoten: 0 | BZ=0000 AC=00000000000 AR=0000 | FE=0 FUE=0
  BS=0 US=1 E=0 A=0 V1=0 V2=0
Rückkehrcode: 4
```

Auch `H G Ü E` endet im Halt: `Ü` stellt den Transport-Befehl auf, `E` (ohne M) führt genau einen Schritt aus (Einzelschritt, Abschnitt 3.3) und hält wieder.

```
$ python3 c8205_interp.py Tests/ue-e.txt ; echo "Rückkehrcode: $?"
=== Test ue-e (#PROT STUFE=0) ===
Ende: Halt (Schalter H): Tastenfolge endet ohne R | Knoten: 0 | BZ=0001 AC=00000000000 AR=0000 | FE=0 FUE=0
  BS=0 US=1 E=0 A=0 V1=0 V2=0
Rückkehrcode: 4
```

3.4 Prüfungen (optional)

Hinter der Tastenfolge kann ein Teil `Prüfungen:` stehen. Er enthält höchstens eine Zeile mit Zuweisungen `SCHLÜSSEL=WERT` für Register und Flags (`BZ`, `AC`, `AR`, `FE`, `FUE`, `BS`, `US`, `E`, `A`, `V1`, `V2`; BZ und AR vierstellig, AC elfstellig, Flags 0 oder 1, jeder Schlüssel höchstens einmal), beliebig viele Speicherzeilen `Adresse: Wert` und – neu – bis zu zwei **Dateiprüfungen**. Nicht genannte Schlüssel werden nicht geprüft.

```
Prüfungen:
BZ=0004 AC=00000000145 US=1
3000: 00000000100
--pruefe-stanzer1 soll.txt
```

Der Interpreter vergleicht nach dem Lauf jeden Wert einzeln und meldet `Test wahr` oder `Test falsch` (Beispiele in Abschnitt 6.5). `--pruefe-stanzer1 DATEI` und `--pruefe-sw-ausgabe DATEI` vergleichen die vom Lauf erzeugte Stanzer- bzw. Schreibwerk-Datei mit der Soll-Datei `DATEI`; ein Name ohne Ordner liegt neben der Testdatei. Jede Zeile steht höchstens einmal; der Schalter gleichen Namens auf der Kommandozeile hat Vorrang. Die Dateiprüfung setzt das zugehörige Gerät voraus (`--stanzer1` bzw. `--sw-ausgabe`, auch als Geräte-Zeile).

```
$ python3 c8205_interp.py Tests/pruefe-ohne.txt ; echo "Rückkehrcode: $?"
=== Test pruefe-ohne (#PROT STUFE=0) ===
FORMALE PRUEFUNG NEGATIV - Test startet nicht, Klaerung erforderlich:
  - --pruefe-stanzer1 ohne --stanzer1: es gibt keine erzeugte Datei, die verglichen werden koennte
Rückkehrcode: 16
```

Dateiprüfung ohne Gerät: formaler Fehler, der Test startet nicht.

> **Hinweis** Bricht der Lauf vorher ab (Stub, Gerätefehler), wird nicht geprüft; die Ausgabe lautet dann „Ergebnisprüfung: nicht durchgeführt“ und der Test gilt nicht automatisch als „falsch“. Ein leerer Abschnitt `Prüfungen:` gilt als fehlender.

3.5 Formale Prüfung vor dem Start

Vor dem Start prüft der Interpreter: Zeilenformat, keine Adresse mit widersprüchlichem Wert, gültige `#SPEI`-Spanne, gültige `#PROT`-Stufe, vorhandene und gültige Tastenfolge, gültiger `Prüfungen:`-Teil sowie die angegebenen Geräte-Pfade (`--leser1`, `--leser2`, `--k0 datei=`, auch als Zeile der Testdatei). Bei einer Unstimmigkeit startet der Test nicht (Rückkehrcode 16) und wird nicht eigenmächtig korrigiert.

```
$ python3 c8205_interp.py Tests/formal.txt ; echo "Rückkehrcode: $?"
=== Test formal (#PROT STUFE=0) ===
FORMALE PRUEFUNG NEGATIV - Test startet nicht, Klaerung erforderlich:
  - Zeile 3: ungueltiges Zeilenformat: '0000: 000000002'
  - Zeile 5: Adresse 0001 widerspruechlich belegt
Rückkehrcode: 16
```

Zeile 3 hat nur neun statt elf Stellen im Wert; Adresse 0001 kommt zweimal mit verschiedenen Werten vor.

3.6 Geräte in der Testdatei

Der Test kann seine Geräte selbst festlegen. Dazu steht je eine Zeile in der Testdatei, an beliebiger Stelle, üblicherweise bei den Testmarken im Teil `Trommeldatei:`. Die Zeilen haben dieselbe Form wie die gleichnamigen Schalter:

```
--leser1 PFAD         # Lochstreifen für Leser 1 (K1): Datei oder Ordner
--leser2 PFAD         # Lochstreifen für Leser 2 (K2): Datei oder Ordner
--stanzer1 PFAD       # Ausgabedatei bzw. -ordner des Stanzers 1 (K1)
--sw-ausgabe PFAD     # Schreibwerk-Ausgabe (K2) nach PFAD/sw-ausgabe.txt
--k0 datei=PFAD       # K0-Eingabe (T und SW) aus Datei oder Ordner
```

- Jede Zeile darf höchstens einmal vorkommen; ein Kommentar `# …` am Zeilenende ist zulässig.
- **Die Kommandozeile hat Vorrang:** Ist derselbe Schalter beim Aufruf angegeben, gilt dessen Wert; die Zeile der Testdatei wird übergangen.
- Ein relativer Pfad gilt vom aktuellen Ordner des Aufrufs aus (wie auf der Kommandozeile), nicht vom Ordner der Testdatei.

- Eine Zeile ohne Wert, eine doppelte Zeile, `--k0` ohne `datei=` und ein nicht vorhandener Leser- oder `--k0`-Pfad sind Formfehler: der Test startet nicht (Rückkehrcode 16; Beispiel in Kapitel 7).
- Zeigt `--leser1` oder `--leser2` auf einen **Ordner**, erscheint ein Hinweis: alle Dateien darin werden als Lochstreifen gelesen; für einen Test ist eine einzelne Datei sicherer.
- Mit der Zeile `--sw-ausgabe SW-Ausgabe` braucht der Aufruf keinen Schalter; am Terminal erscheint nichts, die Zeichen stehen in `SW-Ausgabe/sw-ausgabe.txt`.

Beispiel: Die Testdatei `sw-zeile.txt` wählt K2 an und gibt die Zeichen 5 und Punkt aus; die Zeile `--sw-ausgabe SW-Ausgabe` steht bei den Testmarken. Das Terminal zeigt nur die Endzeile, die Zeichen stehen in der Datei:

```
Trommeldatei:
--sw-ausgabe SW-Ausgabe # Schreibwerk-Ausgabe in Datei
#PROT STUFE=0
0000: 00004500030
0001: 00000250030
0002: 00000730030
0003: 00000000020
Tastenfolge:
H G R
```

```
$ python3 c8205_interp.py Tests/sw-zeile.txt ; echo "Rückkehrcode: $?"
=== Test sw-zeile (#PROT STUFE=0) ===
Ende: Unbedingter Stop (US:=1) | Knoten: 74 | BZ=0004 AC=00000000000 AR=0000 | FE=0 FUE=0 BS=0 US=1 E=0 A=0
  V1=0 V2=0
Rückkehrcode: 4
$ cat SW-Ausgabe/sw-ausgabe.txt
5.
```

4 Interpreter starten

4.1 Kommandozeile

```
python3 c8205_interp.py [--mdverz VERZ] [--out VERZ|DATEI.log]
        [--leser1 PFAD] [--leser2 PFAD] [--stanzer1 PFAD] [--sw-ausgabe PFAD]
        [--k0 datei=PFAD] [--knotentest] [--NO-TERMOUT]
        [--pruefe-stanzer1 DATEI] [--pruefe-sw-ausgabe DATEI]  TEST.txt [TEST2.txt ...]
```

Schalter	Wirkung	Vorgabe
`TEST.txt`	Testdatei(en). Ist der Pfad nicht vorhanden, wird der Dateiname unterhalb des aktuellen Ordners und des Skript-Ordners gesucht (Groß-/Kleinschreibung egal). Bei mehreren Treffern werden sie aufgelistet und der Test startet nicht.	–
`--mdverz VERZ`	Verzeichnis mit den beiden Regelwerk-Dateien `C8205-Interpreter-Eingänge_der_Knoten.md` (Knotengraph) und `C8205-Interpreter-Beschreibung_der_Knoten.md` (Variablentabelle T2); fehlt eine, bricht der Start ab (Rückkehrcode 2).	`markdown-dateien/` neben dem Skript
`--out …`	Mit Dateiendung (z. B. `Test-Protokolle/t1.log`): Protokoll genau in diese Datei (Ordner werden angelegt; nur mit einem Test). Ohne Endung: Ordner, Protokoll dann in `ORDNER/<Testname>/protokoll.txt`.	`Test-Protokolle/` im Skript-Ordner
`--leser1 PFAD --leser2 PFAD`	Lochstreifen für Leser 1 (Kanal K1) bzw. Leser 2 (K2): Datei oder Ordner. Auch als Zeile der Testdatei möglich (Abschnitt 3.6).	nicht angegeben
`--stanzer1 PFAD`	Ausgabedatei des Stanzers 1 (Kanal K1); ein Ordner (auch ein Name mit „/“ am Ende) ergibt `stanzer-ausgabe.txt` darin. Auch als Zeile der Testdatei möglich.	nicht angegeben
`--sw-ausgabe PFAD`	Schreibwerk-Ausgabe (Kanal K2) in die Datei `sw-ausgabe.txt` dieses Verzeichnisses (PFAD ohne Endung: Verzeichnis, wird angelegt; mit Endung: genau diese Datei). Angelegt erst bei der ersten Ausgabe eines Tests, danach angehängt. Auch als Zeile der Testdatei möglich.	nicht angegeben (Ausgabe ans Terminal, dunkelgelb)
`--k0 datei=PFAD`	Eingabequelle für Kanal K0 (T und SW): Datei oder Ordner mit einem Zeichen oder Namen je Zeile. Auch als Zeile der Testdatei möglich.	nicht angegeben (Eingabe vom Terminal)
`--pruefe-stanzer1 DATEI`	Dateiprüfung: vergleicht die über `--stanzer1` erzeugte Datei mit der Soll-Datei DATEI (ein Name ohne Ordner liegt neben der Testdatei). Setzt `--stanzer1` voraus; nur mit **einer** Testdatei. Auch als Zeile im Teil `Prüfungen:` möglich (Abschnitt 3.4).	nicht angegeben
`--pruefe-sw-ausgabe DATEI`	wie `--pruefe-stanzer1`, aber für die über `--sw-ausgabe` erzeugte Datei `sw-ausgabe.txt`; setzt `--sw-ausgabe` voraus.	nicht angegeben
`--knotentest`	Nimmt die Zeilen der durchlaufenen Knoten in das Knotenprotokoll auf. Ändert weder den Ablauf noch die „Knoten“-Zahl der Endzeile.	aus
`--NO-TERMOUT`	Unterdrückt alle Ausgaben des Interpreters selbst am Terminal (Ausnahmen: vom C8205-Programm erzeugte Zeichen und Meldungen bei fehlgeschlagener Vorab-Prüfung). Der Schalter ist case-sensitiv, genau so zu schreiben.	aus
`-h`, `--help`	Hilfetext	–

Die langen Bezeichnungen `--lochbandleser1`, `--lochbandleser2` und `--lochbandstanzer1` sind gleichwertig. Ein nicht angegebenes Gerät ist kein Fehler, solange das Programm es nicht benutzt. Benutzt es das Programm doch, erscheint eine rote Fehlermeldung und der Test endet (Kapitel 7). Ausnahmen: K0-Eingabe (Terminal) und Schreibwerk-Ausgabe (Terminal) haben eine Vorgabe und erzeugen auch ohne Schalter keinen Fehler.

Fehlerhafte Aufrufe zeigen die Kurzhilfe und enden mit Rückkehrcode 2:

```
$ python3 c8205_interp.py --out ergebnis.log Tests/a.txt Tests/b.txt ; echo "Rückkehrcode: $?"
usage: c8205_interp.py [-h] [--mdverz VERZ] [--leser1 PFAD] [--leser2 PFAD]
                       [--stanzer1 PFAD] [--sw-ausgabe PFAD]
                       [--pruefe-stanzer1 DATEI] [--pruefe-sw-ausgabe DATEI]
                       [--k0 datei=PFAD] [--out OUT] [--knotentest]
                       [--NO-TERMOUT]
                       tests [tests ...]
c8205_interp.py: error: --out mit Dateiendung (ergebnis.log) ist nur mit genau einer Testdatei zulaessig
Rückkehrcode: 2
```

Liegen die Regelwerk-Dateien nicht in `markdown-dateien/`, geben Sie das Verzeichnis mit `--mdverz ./markdown-dateien` an; der Wrapper tut das schon (Abschnitt 4.3).

4.2 Mehrere Tests, Protokollort

- Mehrere Testdateien hintereinander: `python3 c8205_interp.py Tests/a.txt Tests/b.txt`. Dann nur `--out` ohne Dateiendung benutzen.
- `--out Test-Protokolle/t1.log` mit mehreren Tests wird abgelehnt (siehe oben, Rückkehrcode 2). Dasselbe gilt für `--pruefe-stanzer1` und `--pruefe-sw-ausgabe`.
- Eine Protokolldatei entsteht nur bei `#PROT STUFE=2`; bei Stufe 0 oder 1 erscheint bei angegebenem `--out` ein Hinweis.

```
$ python3 c8205_interp.py --out ergebnis Tests/kurz2.txt && ls ergebnis/kurz2
=== Test kurz2 (#PROT STUFE=2) ===
Ende: Unbedingter Stop (US:=1) | Knoten: 17 | BZ=0001 AC=00000000000 AR=0000 | FE=0 FUE=0 BS=0 US=1 E=0 A=0
  V1=0 V2=0
Protokolldatei: ergebnis/kurz2/protokoll.txt
```

4.3 Wrapper

Der Wrapper nimmt als erstes Argument eine Testdatei entgegen, leitet sie an den Interpreter und schreibt das Protokoll nach `Test-Protokolle/<Testname>.log` (der Name der Testdatei mit der Endung `.log`). **Alle weiteren Argumente gibt er unverändert an den Interpreter weiter**, sodass jeder Schalter aus Abschnitt 4.1 auch über den Wrapper nutzbar ist. Ohne Argument meldet er „Test-Datei fehlt.“ (Code 1).

```
$ ./c8205_interpreter.sh Tests/0015-030.txt          # Linux/macOS
> c8205_interpreter.bat Tests\0015-030.txt           # Windows

# mit zusätzlichen Schaltern (Testdatei zuerst):
$ ./c8205_interpreter.sh Tests/x.txt --leser1 virtuelle_Geraete/Leser1
$ ./c8205_interpreter.sh Tests/x.txt --leser1 band5.txt --knotentest
$ ./c8205_interpreter.sh Tests/x.txt --mdverz ./markdown-dateien

# Kern des Linux-Wrappers:
shift
python3 c8205_interp.py --mdverz ./markdown-dateien --out "Test-Protokolle/$prot_file" "$file" "$@"
```

Hinweis Die Vorgaben des Wrappers (`--mdverz`, `--out`) stehen vor den weitergereichten Schaltern; bei doppelter Angabe gilt der zuletzt genannte, ein eigenes `--out` überschreibt also die Vorgabe. Der Linux-Wrapper erkennt Schalter mit eigenem Wertargument (`--out`, `--stanzer1`, `--sw-ausgabe`, `--pruefe-stanzer1`, `--pruefe-sw-ausgabe` u. a.); der Wert gilt nie als Testdatei, z. B. `./c8205_interpreter.sh --NO-TERMOUT --stanzer1 virtuelle_Geraete/Stanzer1/ x.test --pruefe-stanzer1 x-stanzer-ausgabe.txt`. Unter Windows (`.bat`) müssen Werte mit Gleichheitszeichen in Anführungszeichen stehen, z. B. `--k0 "datei=band\k0"`; Pfade mit Leerzeichen ebenfalls. Es wird eine Testdatei je Aufruf verarbeitet.

4.4 Rückkehrcodes

Der Interpreter liefert dem Betriebssystem einen Rückkehrcode, der sich stets auf **genau einen** Test bezieht; bei mehreren Testdateien in einem Aufruf zählt der zuletzt bearbeitete. Der Wrapper gibt den Code unverändert weiter. Ob ein Test „wahr“ oder „falsch“ war, lässt sich so ohne Auswertung der Bildschirmzeilen prüfen.

Code	Bedeutung
0	Prüfung definiert (Teil `Prüfungen:` und/oder Dateiprüfung) und Ergebnis **Test wahr**.
4	Lauf ordnungsgemäß beendet, aber **keine Prüfung** angegeben (weder `Prüfungen:` noch `--pruefe-…`).
6	Prüfung definiert, Ergebnis **Test falsch** (auch: Soll- oder Ist-Datei fehlt, Dateiinhalt verschieden).
8	Sonstige Laufzeitfehler: Stub, Gerät nicht nutzbar, Knotenlimit, Abbruch mit Strg+C.
16	Vorab-Fehler, der Test startet nicht: formale Prüfung negativ (u. a. fehlender Geräte-Pfad, `--pruefe-…` ohne Gerät), ungültige Tastenfolge, Testdatei nicht gefunden oder nicht eindeutig.
2	Ungültiger Aufruf: unbekannter oder falsch angegebener Schalter, fehlende Regelwerk-Dateien, `--out` mit Dateiendung bei mehreren Tests. Es startet kein Test.

5 Ein- und Ausgabe

Ein Programm wählt einen Kanal mit einem **Anwahlbefehl** (Operation 030 mit Adresse 4400…4700) aus. Danach wirken Eingabe- und Ausgabebefehle auf das zugehörige Gerät.

Anwahlbefehl	Kanal	Eingabe	Ausgabe	Schalter
`4700 030`	K0	Pult-Tastatur (T) oder Schreibwerk (SW)	keine	`--k0`
`4600 030`	K1	Lochstreifenleser 1	Lochstreifenstanzer 1	`--leser1`, `--stanzer1`
`4500 030`	K2	Lochstreifenleser 2	Schreibwerk (Terminal)	`--leser2`, `--sw-ausgabe`
`4400 030`	K3	frei	frei	– (nicht benutzt)

Ein Ausgabe-Halt-Befehl (`a 030` mit $a_1 \leq 3$) gibt das Zeichen aus, dessen Ausgabe-Adresse (die ersten drei Oktalstellen von a, Kodetabelle 1) passt. Während der Ausgabe ist Flag A = 1. Der Stanzer wertet alle acht Bits aus, das Schreibwerk nur sechs (Spuren v u 8 4 2 1; die Spuren w und p ändern das Zeichen nicht).

> **Achtung** Ein Ausgabe-Halt ohne angewähltes Ausgabegerät (Kanal K0, K3 oder gar keine Anwahl) liegt außerhalb des definierten Betriebs: der Rechner würde mit gesetztem Flag A stoppen, der Interpreter meldet das als **Stub** und lässt A = 1 (Endzeile `A=1`). Dasselbe gilt für eine nicht existierende Anwahl. Kanal K3 hat kein Gerät und wird in dieser Ausbaustufe nie benutzt.

5.1 Schreibwerk-Ausgabe (K2)

- Die Schreibwerk-Ausgabe läuft immer über Kanal K2; die Stellung des Schalters SW hat keinen Einfluss (SW betrifft nur die Eingabeseite, Kanal K0).
- Ohne `--sw-ausgabe` erscheint jedes Zeichen sofort auf dem Terminal in Dunkelgelb.
- Mit `--sw-ausgabe PFAD` erscheint nichts am Terminal; die Zeichen gehen in `PFAD/sw-ausgabe.txt` (bei der ersten Ausgabe eines Tests neu angelegt, danach angehängt). `WRZL` schließt die aktuelle Zeile ab.
- Die Zeichenlage KB/GB wird beim Start **zufällig** gesetzt. `Um. KB` (Ausgabe-Adresse 117x) und `Um. GB` (057x) stellen sie um; sie wirken, geben aber selbst nichts aus. Ohne Umschaltzeichen im Programm ist die Groß-/Kleinschreibung also zufällig.
- Zeichen ohne sichtbare Darstellung erscheinen als `<n>` (z. B. `<Irrung Zeichen>`); `SatzMarke` (272x) ist ein Zeilenumbruch.
- Ist eine Ausgabe-Adresse nicht in Tabelle 1, erscheint nichts (Hinweis im Protokoll).
- Bei STUFE=1 mit `--knotentest` erscheinen die Knotenzeilen erst nach dem Lauf; die Schreibwerk-Ausgabe steht davor.

5.2 Lochstreifenleser (K1, K2)

`--leser1` und `--leser2` nehmen eine **Datei** (ein Streifen) oder einen **Ordner** (Streifen in Namensreihenfolge). Pro Eingabebefehl wird eine Zeile gelesen: acht Zeichen 0/1 für die Spuren `w v u p 8 4 2 1` (die Transportspur wird nicht dargestellt). Leerzeilen sind Vorspann und werden übergangen.

- **Ungerade** Zahl gelochter Spuren = gültig. **Gerade** Zahl = Fehler bei Eingabe: Flag FE := 1, der Lauf hält.
- **Ausnahme „Irrung Zeichen“** (`11111111`, alle acht Spuren): keine Paritätsprüfung, kein FE, nicht ins Adressregister übernommen. Der Eingabebefehl überspringt die Zeile und liest das nächste Zeichen.
- Die Eingabeadresse wird aus dem Lochbild berechnet und muss in Tabelle 1 vorkommen; sie wird mit dem Adressregister ODER-verknüpft.
- Danach wird der Befehl umgestellt: bei Sprung-Umstellung „ja“ (z. B. `:`, `.`, `,`, Tabulator, Um. KB, Um. GB) zu einem unbedingten Sprung an `a ODER Eingabeadresse`; bei „nein“ (Ziffern, Buchstaben) zu einem Lesebefehl, der den **Inhalt der Zelle** `a ODER Eingabeadresse` zum AC addiert (nicht die Eingabeadresse selbst).
- Ist der Streifen zu Ende, wartet der Interpreter still (ohne Meldung) auf eine neu in den Pfad kopierte Streifendatei (der reale Leser würde ebenfalls warten); ein nicht vorhandener Pfad verhindert den Teststart. Strg+C beendet das Warten (Rückkehrcode 8).

> **Achtung** Die Ziffer 5 liefert **nicht** den Wert 5 im AC, sondern den Inhalt der Zelle 0005 (ODER Adressteil des Eingabebefehls). Ist die Zelle nicht belegt, bleibt der AC 0. Legen Sie die gewünschten Konstanten in der Trommeldatei an (Beispiel in Abschnitt 8.3).

5.3 Lochstreifenstanzer (K1)

`--stanzer1 PFAD` ist die Ausgabedatei (bei einem Ordner: `stanzer-ausgabe.txt` darin). Je gestanztem Zeichen entsteht eine Zeile mit acht Zeichen 0/1 für `w v u p 8 4 2 1`. Die Datei wird beim ersten gestanzten Zeichen eines Laufs **neu angelegt** (nicht angehängt) und nur angelegt, wenn wirklich gestanzt wird. Anders als beim Schreibwerk gibt es **keinen** Terminal-Ausweg: ohne `--stanzer1` endet ein Test, der auf K1 ausgibt, mit der roten Fehlermeldung aus Kapitel 7.

5.4 Pult-Tastatur und Schreibwerk-Eingabe (K0)

Ohne `--k0` liest der Interpreter einen Tastendruck je Eingabebefehl vom Terminal (ohne Echo, ohne Enter; Strg+C wirkt). `--k0 datei=PFAD` (oder die gleichnamige Zeile der Testdatei) liest eine Textdatei mit einem Zeichen oder Namen je Zeile (`ZR`, `WM`, `IS`, `IB`, `TAB`, `WRZL` oder Namen aus Tabelle 1; Zeilen mit `#` sind Kommentar). PFAD darf ein Ordner mit mehreren Dateien sein, die in alphanumerischer Reihenfolge gelesen werden. Ist die Eingabe zu Ende, wartet der Interpreter still auf eine neu in den Pfad kopierte Datei; ein nicht vorhandener Pfad verhindert den Teststart.

Zustand	Eingebbar
SW nicht gedrückt (Flag T, Pult-Tastatur)	nur `0`–`9`, `+`, `-`, `,`, `.`, `:`, `ZR`, `WM`, `IS`, `IB` (19 Codes). B und D haben keinen Code und sind nicht eingebbar.
SW gedrückt (Flag SW, Schreibwerk)	alle Zeichen der Tabelle 1

Ein nicht erlaubtes Zeichen wird bei `datei=` mit roter Meldung abgelehnt (der Test endet); bei Terminal-Eingabe wird der Tastendruck ignoriert und auf die nächste gültige Taste gewartet. Zeichen werden über die Spalten „Ascii KB/GB“ der Kodetabelle gefunden; die Eingabe-Zeichenlage (Start KB, geändert durch gelesenes Um. KB/Um. GB) bestimmt, welche Spalte zuerst durchsucht wird. Nur bei Terminal-Eingabe ohne weitere Zeichen (z. B. geschlossenes stdin) endet der Lauf mit `EINGABE_ENDE`.

> **Hinweis** Unter Linux/macOS wird termios, unter Windows msvcrt.getch() benutzt. Unter Windows kann Strg+C in Ausnahmefällen als normales Zeichen ankommen; im Zweifel hilft Strg+Pause oder das Schließen des Konsolenfensters.

6 Ergebnisse lesen

6.1 Bildschirmausgabe

Je Test erscheinen der Reihe nach: die Kopfzeile `=== Test <name> (#PROT STUFE=n) ===`, die Schreibwerk-Zeichen des Programms, eine eventuelle Diagnose (`-- Diagnose --`), bei Stufe 1 und `--knotentest` die Knotenzeilen, die Endzeile und – falls der Test Prüfungen enthält – die Ergebnisprüfung.

6.2 Die Endzeile

```
Ende: <Grund> | Knoten: n | BZ=… AC=… AR=… | FE=0 FUE=0 BS=0 US=1 E=0 A=0 V1=0 V2=0
```

Teil	Bedeutung
`<Grund>`	warum der Lauf endete (Tabelle unten)
`Knoten: n`	Zahl der durchlaufenen Knoten (fassungsabhängig, kein Sollwert, keine Zeitangabe)
`BZ, AR`	Befehlszähler und Adressregister, vierstellig oktal
`AC`	Akkumulator, elfstellig oktal
`FE`	Fehler bei Eingabe
`FUE`	Überlauf (FÜ)
`BS, US`	bedingter bzw. unbedingter Stopp
`E, A`	Eingabe- bzw. Ausgabebefehl aufgestellt (der Lauf wartet noch auf ein Gerät; bei einem Ausgabe-Stub bleibt A = 1)
`V1, V2`	Vergleicher (V3 = V1 und V2)

Endgrund	Bedeutung
Unbedingter Stop (US:=1)	Programm hat ordnungsgemäß gehalten
Bedingter Stop (BS:=1)	bedingter Stopp-Befehl bei eingerastetem Schalter BS
Halt wegen Überlauf (FÜ bereits 1)	Überlauf-Halt; weder US noch BS gesetzt, nur FÜ
Fehler bei Eingabe (FE)	gerade Zahl von Lochungen gelesen
EINGABE_ENDE	Terminal-Eingabe (K0) ohne weitere Zeichen, z. B. geschlossenes stdin. Bei Leser und `--k0 datei=` gibt es kein Ende, sondern stilles Warten.
Halt (Schalter H): Tastenfolge endet ohne R	Tastenfolge endete im Halt (gültig, z. B. `H G Ü E`)
Geraet nicht nutzbar …	Gerät nicht angegeben oder Datei/Zeichen ungültig (rote Meldung davor); die Zeile „TEST NICHT ORDNUNGSGEMAESS BEENDET“ steht ebenfalls davor. Gilt nicht für K0 (Terminal) und K2-Ausgabe (Terminal).
STUB bei Knoten n …	Regelwerk unvollständig oder Kanal K3 benutzt, Ausgabe-Halt ohne Gerät (Kapitel 7)
Abbruch: Knotenlimit erreicht (Endlosschleife?)	der Lauf hat die Sicherheitsgrenze von über 2 Millionen Knoten überschritten: das Programm hält nie an (Kapitel 7)
Tastenfolge nicht gueltig/definiert	Test startete nicht (Abschnitt 3.3)

Beispiel für den Endgrund „Halt wegen Überlauf“: Der Test addiert zweimal `1` zu `37777777777`; nach dem Überlauf hält der Rechner, es ist weder US noch BS gesetzt, nur FUE=1.

```
$ python3 c8205_interp.py Tests/ueberlauf.txt ; echo "Rückkehrcode: $?"
=== Test ueberlauf (#PROT STUFE=0) ===
Ende: Halt wegen Ueberlauf (FUE bereits 1) | Knoten: 71 | BZ=0003 AC=40000000000 AR=3001 | FE=0 FUE=1 BS=0
  US=0 E=0 A=0 V1=0 V2=0
Rückkehrcode: 4
```

6.3 Diagnose bei Befehl 164

Der Leerbefehl 164 löst die Diagnose aus, wenn für seine Adresse xxxx Testmarken definiert sind. Reihenfolge: zuerst die `#DOK`-Texte (Dateireihenfolge), dann alle `#SPEI`-Bereiche, dann alle `#SPUR`-Spuren; jede Speicherzelle steht als `aaaa: wwwwwwwwwww`. Vor `#SPEI`/`#SPUR` steht die auslösende Marke selbst als Zeile. Die Marken sind unabhängig voneinander (auch ein einzelnes `#SPEI` ohne `#DOK` gibt seine Zellen aus); passt keine Marke, entfällt die Diagnose ganz. Ein Beispiel mit `#SPEI` zeigt Kapitel 8.4.

> **Achtung** Die Leer-Befehle 164 und 166 laufen **ohne Resultatbehandlung**: das Z-Feld erzeugt keine Verschiebung des AC.

6.4 Knotenprotokoll und Protokolldatei

Bei `#PROT STUFE=1` erscheinen die Knotenzeilen auf dem Bildschirm, bei STUFE=2 in der Protokolldatei – aber nur, wenn Sie mit `--knotentest` aufrufen. Ohne den Schalter entfallen die Zeilen `Kxxx …`; erhalten bleiben die `BR=`-Zeilen (vor Knoten 2 und 7), die Diagnose sowie Fehler- und Stub-Meldungen.

```
$ python3 c8205_interp.py --knotentest Tests/kurz.txt
=== Test kurz (#PROT STUFE=1) ===
-- Knotenzustaende --
K001 KON B BZ=0000 AR=0000 AC=00000000000
  BR=0000 000  (vor Knoten 002)  BZ=0000 AC=00000000000 FÜ=0 V1=0 V2=0 V3=0
K002 PHA Phase I BZ=0000 AR=0000 AC=00000000000
K003 ANW 001 BZ=0001 AR=0000 AC=00000000000
K004 TST 002 BZ=0001 AR=0000 AC=00000000000 -> falsch
K005 TST 003 BZ=0001 AR=0000 AC=00000000000 -> wahr
K008 PHA Phase II BZ=0001 AR=0000 AC=00000000000
K009 ANW 005 BZ=0001 AR=0000 AC=00000000000
K010 TST 006 BZ=0001 AR=0000 AC=00000000000 -> falsch
K011 ANW 007 BZ=3000 AR=0000 AC=00000000000
K012 TST 008 BZ=3000 AR=0000 AC=00000000000 -> falsch
K015 PHA Phase III BZ=3000 AR=0000 AC=00000000000
…
```

Auszug: die ersten 14 Zeilen der Ausgabe.

Zeile	Lesart
`K009 ANW 005 BZ=0001 AR=0001 AC=…`	Knoten 9, Typ Anweisung (ANW), ANWEISUNG-005; danach BZ, AR und AC, jeweils oktal
`K004 TST 002 … -> falsch`	Knoten 4, Typ Test (TST), ANWEISUNG-002; `-> wahr` oder `-> falsch` ist das Testergebnis
`KON`, `PHA`	Konnektor bzw. Phasenzuordnung („Phase I“ … „Phase IV“)
`BR=0000 000 (vor Knoten 002) BZ=… AC=… FÜ=0 V1=0 V2=0 V3=0`	Befehlsregister (Adressteil und Operationsteil des geladenen Befehls) vor Beginn von Phase I; dazu BZ, AC, Überlauf FÜ und die Vergleicher

Die Protokolldatei (nur bei STUFE=2) hat bis zu vier Abschnitte: `== 1. Ausgangsbedingungen ==` (übernommene Trommel, Tastenfolge mit der Wirkung jeder Taste), `== 2. Zustaende des Knotenautomaten ==`, `== Ende ==` und, falls der Test Prüfungen enthält, `== 3. Ergebnispruefung ==`. Beispiel einer Tastenfolge mit vielen Token (aus Abschnitt 8.2):

```
Test tf-demo

== 1. Ausgangsbedingungen ==
Trommeldatei (uebernommen, Ausschnitt; nicht genannte Zellen = 0):
  0000: 00000000020
  #PROT STUFE=2
Tastenfolge: SW=1 SW=0 BS=1 SP=1 H G Ü ST BS=0 R
  SW gedrueckt: Schalter SW -> Flag SW := 1, T := 0
  SW geloest (Stellung T): Flag SW := 0, T := 1
  BS : Schalter BS gedrueckt (eingerastet) -> Flag BS := 1
  SP : Schalter SP gedrueckt (eingerastet) -> Flag SP := 1
  H  : Schalter H-R -> "H", Flag H := 1
  G  : alle Flags ausser H,T,BS,SP := 0 (BS, SP: eingerastete Schalter); BR := 0000 000 (AR:=0000, OR:=000);
  US := 1; K0 angewaehlt, SW nicht gedrueckt -> T := 1; <BZ> := 0 (vorweggenommene Wirkung des Sprungs nach
  0000)
  Ue : AR := 7777; OR := OR ODER 240 -> BR = 7777 240 (Transport, aufgestellt, nicht ausgefuehrt)
  ST : (US oder BS) OR := OR ODER 240 -> BR = 7777 240, AR unveraendert; US := 0 (aufgestellt, nicht
  ausgefuehrt)
  BS : Schalter BS geloest -> Flag BS := 0
  R  : Schalter H-R -> "R", Flag H := 0, US := 0; Interpretation beginnt bei Konnektor B bzw. bei Knoten 19,
  wenn BR aufgestellt ist
  Nicht spezifizierte Register (AC, ...) starten mit 0.

== 2. Zustaende des Knotenautomaten ==
  BR=7777 240  (vor Knoten 002)  BZ=0000 AC=00000000000 FÜ=0 V1=0 V2=0 V3=0

== Ende ==
Unbedingter Stop (US:=1)
BZ=0001 AC=00000000000 AR=0000 FE=0 FUE=0 BS=0 US=1 E=0 A=0 V1=0 V2=0
```

6.5 Ergebnisprüfung

Hat die Testdatei einen `Prüfungen:`-Teil und endete der Lauf ordnungsgemäß, folgt auf die Endzeile die Prüfung: je Wert Soll und Ist mit OK oder FEHLER, danach `Test wahr` oder `Test falsch`. Sie erscheint unabhängig von `#PROT STUFE`; bei STUFE=2 steht dasselbe als Abschnitt `== 3. Ergebnispruefung ==` am Ende der Protokolldatei.

```
$ python3 c8205_interp.py Tests/pruef-falsch.txt ; echo "Rückkehrcode: $?"
=== Test pruef-falsch (#PROT STUFE=0) ===
-- Diagnose --
#DOK-0001 Test Ladebefehl und Addition
Ende: Unbedingter Stop (US:=1) | Knoten: 90 | BZ=0004 AC=00000000145 AR=0000 | FE=0 FUE=0 BS=0 US=1 E=0 A=0
  V1=0 V2=0
Ergebnispruefung:
  BZ: soll=0003 ist=0004 FEHLER
  AC: soll=00000000145 ist=00000000145 OK
Test falsch
Rückkehrcode: 6
```

`Test wahr` gibt es nur, wenn **alle** angegebenen Prüfungen übereinstimmen. Hier stimmt BZ nicht (soll 0003, ist 0004); der AC stimmt. Rückkehrcode 6 (Abschnitt 4.4). Beispiel mit Erfolg: Abschnitt 8.4.

Dateiprüfungen

`--pruefe-stanzer1` und `--pruefe-sw-ausgabe` erscheinen im selben Block hinter den Einzelprüfungen, je Prüfung eine Zeile:

Zeile	Bedeutung
`Stanzer 1: soll=<Soll-Datei> ist=<Ist-Datei> OK`	Inhalt (zeilenweise) gleich; für das Schreibwerk steht `Schreibwerk:` davor.
`… FEHLER (Soll-Datei fehlt)`	die Soll-Datei existiert nicht (Name ohne Ordner: neben der Testdatei suchen)
`… FEHLER (Ist-Datei fehlt, vom Lauf nicht angelegt)`	der Lauf hat nichts gestanzt bzw. ausgegeben; eine übrig gebliebene Datei eines früheren Laufs zählt nicht
`… FEHLER (Inhalt verschieden, diff):`	darunter der Unterschied im Format `diff -u` (Soll gegen Ist, höchstens 100 Zeilen, danach „… n weitere diff-Zeilen nicht angezeigt“)

Beispiel: Der Stanzer-Test aus Abschnitt 8.3 gegen eine Soll-Datei, die in der letzten Zeile abweicht:

```
$ python3 c8205_interp.py --pruefe-stanzer1 soll-falsch.txt Tests/stanzer-pruefe.txt ; echo "Rückkehrcode: $?"
=== Test stanzer-pruefe (#PROT STUFE=0) ===
Ende: Unbedingter Stop (US:=1) | Knoten: 74 | BZ=0004 AC=00000000000 AR=0000 | FE=0 FUE=0 BS=0 US=1 E=0 A=0
  V1=0 V2=0
Ergebnispruefung:
  US: soll=1 ist=1 OK
  Stanzer 1: soll=Tests/soll-falsch.txt ist=out/band.txt FEHLER (Inhalt verschieden, diff):
    --- soll Tests/soll-falsch.txt
    +++ ist out/band.txt
    @@ -1,2 +1,2 @@
     00010101
    -00111111
    +00111011
Test falsch
Rückkehrcode: 6
```

Der Rückkehrcode folgt dem Gesamtergebnis (0 oder 6). Mit `--NO-TERMOUT` entfällt die Bildschirmausgabe; das Ergebnis steht dann bei `#PROT STUFE=2` in der Protokolldatei und im Rückkehrcode.

7 Wenn etwas schiefgeht

Gerätefehler erscheinen **in Rot** auf dem Terminal (Standardfehlerausgabe, nur mit Terminal gefärbt) und beginnen mit „FEHLER:“. Danach folgt die Zeile „TEST NICHT ORDNUNGSGEMAESS BEENDET: …“ und die Endzeile. Das Fehlerverzeichnis ist nach dem Meldungstext sortiert; die letzte Spalte nennt den Rückkehrcode (Abschnitt 4.4).

Meldung (Anfang)	Ursache	Abhilfe	Code
Abbruch: Knotenlimit erreicht (Endlosschleife?)	Programm läuft ohne Stopp-Befehl weiter, z. B. weil eine leere Zelle (Wert 0 = Sprung nach 0000) erreicht wird	Stopp-Befehl `00000000020` hinter das Programmende schreiben; Programmfluss prüfen	8
Ausgabe auf Lochbandstanzer 1 (Kanal K1) angefordert, aber --stanzer1 nicht angegeben.	Programm stanzt, Schalter fehlt	`--stanzer1 PFAD` angeben (Schalter oder Zeile)	8
Eingabe von Lochbandleser n (Kanal Kn) angefordert, aber --lesern nicht angegeben.	Programm liest von K1/K2, Schalter fehlt	`--leser1` bzw. `--leser2` angeben (Schalter oder Zeile)	8
FORMALE PRUEFUNG NEGATIV …	Testdatei formal fehlerhaft (Zeilenformat, doppelte Adresse, `#SPEI`-Spanne, `#PROT`, Tastenfolge fehlt, `Prüfungen:`, Geräte-Pfad)	die genannte Zeile korrigieren (Abschnitt 3.5)	16
K0-Datei nicht lesbar: …	Datei fehlt oder nicht lesbar	Pfad prüfen	8
K0-Eingabe: '…' ist an der Pult-Tastatur (T) nicht eingebbar …	Zeichen ist an T nicht erlaubt (z. B. B, Buchstaben)	`SW=1` vor `G` in die Tastenfolge oder ein anderes Zeichen	8
--k0: 'datei=PFAD' erwartet …	ungültiger Wert für `--k0` (Schalter: Code 2; Zeile n der Testdatei: Code 16)	`--k0 datei=PFAD` schreiben	2 / 16
K0: Eingabe-Pfad nicht gefunden: …	bei `--k0 datei=PFAD` (Schalter oder Zeile) existiert der Pfad nicht	Pfad prüfen	16
Knotenbeschreibung nicht gefunden: … / Knotengraph nicht gefunden: …	eine der beiden Regelwerk-Dateien fehlt im `--mdverz`-Verzeichnis	`--mdverz` korrekt angeben oder Datei dorthin legen	2
Lochbandleser n: Lochstreifen-Pfad nicht gefunden: …	Pfad falsch	Pfad prüfen	16
Lochstreifen …: ungueltige Zeile …	Zeile ist nicht acht Zeichen aus 0/1	Streifendatei korrigieren	8
--out mit Dateiendung … ist nur mit genau einer Testdatei zulaessig	`--out x.log` zusammen mit mehreren Tests	Ordner ohne Endung verwenden oder Tests einzeln starten	2
--pruefe-stanzer1 ohne --stanzer1: … / --pruefe-sw-ausgabe ohne --sw-ausgabe: …	Dateiprüfung ohne das zugehörige Gerät; es gäbe keine Ist-Datei	passenden Geräteschalter ergänzen (Schalter oder Zeile der Testdatei)	16
--pruefe-stanzer1/--pruefe-sw-ausgabe sind nur mit genau einer Testdatei zulaessig	Dateiprüfung mit mehreren Testdateien	Tests einzeln starten	2
STUB bei Knoten n: …	eine vom Programm benutzte Funktion ist nicht umgesetzt oder liegt außerhalb des definierten Betriebs (siehe unten)	Meldung und Testdatei notieren; Programm prüfen	8
Tastenfolge nicht gueltig/definiert	unbekanntes Token, weder R noch Halt am Ende, M nicht losgelassen	Abschnitt 3.3	16
Testdatei nicht gefunden: …	Name weder als Pfad noch unterhalb der Suchordner vorhanden	Pfad, Schreibweise und aktuellen Ordner prüfen (wird mit angezeigt)	16
Testdatei … nicht eindeutig gefunden	mehrere Dateien gleichen Namens	vollständigen Pfad angeben	16
Zeile n: --leser1/--leser2/--stanzer1/--sw-ausgabe/--k0 …	Formfehler einer Geräte-Zeile der Testdatei: ohne Wert, mehrfach, `--k0` ohne `datei=`	Zeile korrigieren; jede Zeile höchstens einmal (Abschnitt 3.6)	16

Stub. Ein Stub ist kein Fehler des Aufrufs, sondern die Meldung „diese Stelle des Regelwerks ist nicht umgesetzt“ bzw. „außerhalb des definierten Betriebs“. Für die 512 Operationscodes gibt es keinen Stub mehr. Ein Stub tritt auf, wenn ein Programm auf **Kanal K3** ein- oder ausgibt, wenn ein **Ausgabe-Halt ohne Ausgabegerät** ausgeführt wird (Kanal K0, K3 oder keine Anwahl; A bleibt 1), wenn ein Anwahlbefehl mit $a_2 < 4$

benutzt wird oder wenn eine Eingabeadresse in Tabelle 1 nicht vorkommt. Notieren Sie Meldung und Testdatei; der Test wurde nicht „repariert“.

Beispiele

Gerät nicht angegeben

Der Test aus Abschnitt 8.3 liest von Leser 1; ohne `--leser1` endet er mit der roten Meldung, Rückkehrcode 8:

```
$ python3 c8205_interp.py Tests/leser-ziffer.txt ; echo "Rückkehrcode: $?"
=== Test leser-ziffer (#PROT STUFE=0) ===
FEHLER: Eingabe von Lochbandleser 1 (Kanal K1) angefordert, aber --leser1 nicht angegeben.
TEST NICHT ORDNUNGSGEMAESS BEENDET: Geraet nicht nutzbar (Knoten 48): --leser1 nicht angegeben
Ende: Geraet nicht nutzbar (Knoten 48): --leser1 nicht angegeben | Knoten: 37 | BZ=0002 AC=00000000000 AR=0000
   | FE=0 FUE=0 BS=0 US=0 E=1 A=0 V1=0 V2=0
Rückkehrcode: 8
```

Entsprechend für den Stanzer (Test `stanzer-zeichen.txt`, Abschnitt 8.3) ohne `--stanzer1`; der Ausgabebefehl steht aufgestellt (A=1):

```
$ python3 c8205_interp.py Tests/stanzer-zeichen.txt ; echo "Rückkehrcode: $?"
=== Test stanzer-zeichen (#PROT STUFE=0) ===
FEHLER: Ausgabe auf Lochbandstanzer 1 (Kanal K1) angefordert, aber --stanzer1 nicht angegeben.
TEST NICHT ORDNUNGSGEMAESS BEENDET: Geraet nicht nutzbar (Knoten 23): --stanzer1 nicht angegeben
Ende: Geraet nicht nutzbar (Knoten 23): --stanzer1 nicht angegeben | Knoten: 38 | BZ=0002 AC=00000000000
  AR=0250 | FE=0 FUE=0 BS=0 US=0 E=0 A=1 V1=0 V2=0
Rückkehrcode: 8
```

Zeichen an der Pult-Tastatur nicht eingebbar

```
$ python3 c8205_interp.py --k0 datei=band/k0b.txt Tests/k0-ziffer.txt ; echo "Rückkehrcode: $?"
=== Test k0-ziffer (#PROT STUFE=0) ===
FEHLER: K0-Eingabe: 'B' ist an der Pult-Tastatur (T) nicht eingebbar - nur 0-9 + - , . : ZR WM IS IB (fuer
  alle Codes: Schalter SW druecken).
TEST NICHT ORDNUNGSGEMAESS BEENDET: Geraet nicht nutzbar (Knoten 48): 'B' ist an der Pult-Tastatur (T) nicht
  eingebbar - nur 0-9 + - , . : ZR WM IS IB (fuer alle Codes: Schalter SW druecken)
Ende: Geraet nicht nutzbar (Knoten 48): 'B' ist an der Pult-Tastatur (T) nicht eingebbar - nur 0-9 + - , . :
  ZR WM IS IB (fuer alle Codes: Schalter SW druecken) | Knoten: 37 | BZ=0002 AC=00000000000 AR=0000 | FE=0
  FUE=0 BS=0 US=0 E=1 A=0 V1=0 V2=0
Rückkehrcode: 8
```

Stub: Ausgabe ohne angewähltes Gerät

Das Programm gibt die Ziffer 5 aus (`0000: 00000250030`), ohne vorher einen Kanal anzuwählen. Nach dem Original hält der Rechner mit Flag A; der Interpreter meldet das als Stub und lässt A = 1:

```
$ python3 c8205_interp.py Tests/stub-ausgabe.txt ; echo "Rückkehrcode: $?"
=== Test stub-ausgabe (#PROT STUFE=0) ===
TEST NICHT ORDNUNGSGEMAESS BEENDET: STUB bei Knoten 23: Ausgabe-Halt ohne Ausgabegeraet (Kanal K0, K3 oder
  keine Anwahl); nach der Originaldokumentation 4.3.2.5 stoppt der Rechner mit gesetztem Flag A (Lampe A)
Ende: STUB bei Knoten 23: Ausgabe-Halt ohne Ausgabegeraet (Kanal K0, K3 oder keine Anwahl); nach der
  Originaldokumentation 4.3.2.5 stoppt der Rechner mit gesetztem Flag A (Lampe A) | Knoten: 19 | BZ=0001
  AC=00000000000 AR=0250 | FE=0 FUE=0 BS=0 US=0 E=0 A=1 V1=0 V2=0
Rückkehrcode: 8
```

Tastenfolge ungültig

Hier fehlt vor `M-` ein `M+` (Test `tf-m.txt`, Tastenfolge `H G M- R`):

```
$ python3 c8205_interp.py Tests/tf-m.txt ; echo "Rückkehrcode: $?"
=== Test tf-m (#PROT STUFE=0) ===
TEST NICHT ORDNUNGSGEMAESS BEENDET: Tastenfolge nicht gueltig/definiert
Ende: Tastenfolge nicht gueltig/definiert | Knoten: 0 | BZ=0000 AC=00000000000 AR=0000 | FE=0 FUE=0 BS=0 US=0
  E=0 A=0 V1=0 V2=0
Rückkehrcode: 16
```

Programm ohne Stopp (Endlosschleife)

Das Programm lädt eine Zahl und läuft dann in die leere Zelle 0001 (Wert 0). Ein Befehl mit dem Wert 0 ist der unbedingte Sprung nach Adresse 0000: das Programm beginnt von vorn und endet nie. Nach über zwei Millionen Knoten bricht der Interpreter ab.

```
$ python3 c8205_interp.py Tests/endlos.txt ; echo "Rückkehrcode: $?"
=== Test endlos (#PROT STUFE=0) ===
TEST NICHT ORDNUNGSGEMAESS BEENDET: Abbruch: Knotenlimit erreicht (Endlosschleife?)
Ende: Abbruch: Knotenlimit erreicht (Endlosschleife?) | Knoten: 2000001 | BZ=0001 AC=00000000000 AR=3000 |
  FE=0 FUE=0 BS=0 US=0 E=0 A=0 V1=0 V2=0
Rückkehrcode: 8
```

Geräte-Zeile ohne Wert

Die Testdatei enthält in Zeile 3 die Zeile `--stanzer1` ohne Pfad:

```
$ python3 c8205_interp.py Tests/geraet-leer.txt ; echo "Rückkehrcode: $?"
=== Test geraet-leer (#PROT STUFE=0) ===
FORMALE PRUEFUNG NEGATIV - Test startet nicht, Klaerung erforderlich:
  - Zeile 3: --stanzer1 ohne Wert
Rückkehrcode: 16
```

Testdatei nicht gefunden

```
$ python3 c8205_interp.py Tests/gibtsnicht.txt ; echo "Rückkehrcode: $?"
FEHLER: Testdatei nicht gefunden: 'Tests/gibtsnicht.txt' (aktuelles Verzeichnis: /home/claude/c8205)
Rückkehrcode: 16
```

Pfad der K0-Eingabe nicht vorhanden

Ein nicht vorhandener `--leser1`-, `--leser2`- oder `--k0`-Pfad wird schon vor dem Start gemeldet, auch wenn er als Zeile der Testdatei steht:

```
$ python3 c8205_interp.py --k0 datei=band/gibtsnicht.txt Tests/a.txt ; echo "Rückkehrcode: $?"
=== Test a (#PROT STUFE=0) ===
FORMALE PRUEFUNG NEGATIV - Test startet nicht, Klaerung erforderlich:
  - K0: Eingabe-Pfad nicht gefunden: band/gibtsnicht.txt
Rückkehrcode: 16
```

Mit --NO-TERMOUT bleiben Vorab-Meldungen sichtbar

```
$ python3 c8205_interp.py --NO-TERMOUT Tests/formal.txt ; echo "Rückkehrcode: $?"
FORMALE PRUEFUNG NEGATIV - Test startet nicht, Klaerung erforderlich:
  - Zeile 3: ungueltiges Zeilenformat: '0000: 000000002'
  - Zeile 5: Adresse 0001 widerspruechlich belegt
Rückkehrcode: 16
```

Die Kopfzeile „=== Test ... ===“ entfällt, die Meldung der formalen Prüfung bleibt.

8 Beispiele Schritt für Schritt

Alle Beispiele sind lauffähig und mit der auf der Titelseite genannten Fassung ausgeführt. Legen Sie die Dateien wie angegeben an; die Ordner `Tests/` und `band/` im Projektordner sind frei wählbar. Die Zahl hinter „Knoten:“ ist fassungsabhängig.

8.1 Ausgabe auf dem Schreibwerk

Das Beispiel steht vollständig in Abschnitt 2.3: Testdatei `Tests/0015-030.txt`, Aufruf mit dem Wrapper, Ausgabe „Jahr 1972“ in Dunkelgelb. Verwendet werden die Token `H`, `G`, `M-drücken`, `E`, `M-loslassen`, `R` und die Marken `#DOK`, `#PROT STUFE=2`. Mit `--sw-ausgabe` erscheint der Text nicht am Terminal, sondern in `SW-Ausgabe/sw-ausgabe.txt`; mit `--NO-TERMOUT` zusätzlich keine Zeile des Interpreters:

```
$ python3 c8205_interp.py --NO-TERMOUT --sw-ausgabe SW-Ausgabe Tests/0015-030.txt
$ cat SW-Ausgabe/sw-ausgabe.txt
Jahr 1972
```

8.2 Alle Bedientasten in einer Tastenfolge

Die Tastenfolge `SW=1 SW=0 BS=1 SP=1 H G Ü ST BS=0 R` benutzt die Token SW, BS, SP, H, G, Ü, ST und R. Das Programm besteht nur aus einem Stopp-Befehl; wichtig ist das Protokoll (Stufe 2), das die Wirkung jedes Tokens zeigt.

```
Trommeldatei:
#PROT STUFE=2
0000: 00000000020
Tastenfolge:
SW=1 SW=0 BS=1 SP=1 H G Ü ST BS=0 R
```

```
$ python3 c8205_interp.py --out Test-Protokolle/tf-demo.log Tests/tf-demo.txt
=== Test tf-demo (#PROT STUFE=2) ===
Ende: Unbedingter Stop (US:=1) | Knoten: 28 | BZ=0001 AC=00000000000 AR=0000 | FE=0 FUE=0 BS=0 US=1 E=0 A=0
  V1=0 V2=0
Protokolldatei: Test-Protokolle/tf-demo.log
```

Das Protokoll (Abschnitt 6.4) zeigt: ST wirkt hier, weil G zuvor US gesetzt hat (Verriegelung). Durch Ü und ST steht BR=7777 240 aufgestellt; R startet deshalb bei Knoten 19, führt den Transport aus und hält danach.

Kanaltasten K0 bis K3

Die Kanaltasten stehen nach `G`. Die Tastenfolge `H G M K0 K1 K2 K3` endet im Halt (kein `R`); das Protokoll zeigt, was jede Taste einstellt. Ein im Programm ausgeführter Anwahlbefehl (4400…4700 030) übersteuert die Einstellung später.

```
$ python3 c8205_interp.py --out Test-Protokolle/kanal-demo.log Tests/kanal-demo.txt
=== Test kanal-demo (#PROT STUFE=2) ===
Ende: Halt (Schalter H): Tastenfolge endet ohne R | Knoten: 0 | BZ=0000 AC=00000000000 AR=0000 | FE=0 FUE=0
  BS=0 US=1 E=0 A=0 V1=0 V2=0
Protokolldatei: Test-Protokolle/kanal-demo.log
$ cat Test-Protokolle/kanal-demo.log
Test kanal-demo

== 1. Ausgangsbedingungen ==
Trommeldatei (uebernommen, Ausschnitt; nicht genannte Zellen = 0):
  0000: 00000000020
  #PROT STUFE=2
Tastenfolge: H G M K0 K1 K2 K3
  H  : Schalter H-R -> "H", Flag H := 1
  G  : alle Flags ausser H,T,BS,SP := 0 (BS, SP: eingerastete Schalter); BR := 0000 000 (AR:=0000, OR:=000);
  US := 1; K0 angewaehlt, SW nicht gedrueckt -> T := 1; <BZ> := 0 (vorweggenommene Wirkung des Sprungs nach
  0000)
  M  : (= M gedrueckt, sofort losgelassen) OR := OR ODER 025 -> BR = AR 025
  K0 : Kanaltaste (rastend): Kanal K0 angewaehlt (SW nicht gedrueckt -> T := 1); ein im Programm ausgefuehrter
   Anwahlbefehl uebersteuert diese Einstellung
  K1 : Kanaltaste (rastend): Kanal K1 angewaehlt; ein im Programm ausgefuehrter Anwahlbefehl uebersteuert
  diese Einstellung
  K2 : Kanaltaste (rastend): Kanal K2 angewaehlt; ein im Programm ausgefuehrter Anwahlbefehl uebersteuert
  diese Einstellung
  K3 : Kanaltaste (rastend): Kanal K3 angewaehlt; ein im Programm ausgefuehrter Anwahlbefehl uebersteuert
  diese Einstellung
  Ende der Tastenfolge ohne R: Rechner bleibt im Halt (H)
  Nicht spezifizierte Register (AC, ...) starten mit 0.
```

```
== 2. Zustaende des Knotenautomaten ==


== Ende ==
Halt (Schalter H): Tastenfolge endet ohne R
BZ=0000 AC=00000000000 AR=0000 FE=0 FUE=0 BS=0 US=1 E=0 A=0 V1=0 V2=0
```

8.3 Zeichen vom Lochstreifen lesen und auf den Stanzer ausgeben

Leser: Die Ziffer 5

Programm: `0000: 00004600030` (K1 anwählen), `0001: 00000000011` (Eingabe-Halt mit Löschen), `0002: 00000000020` (Stopp), dazu die Konstante `0005: 00000000042`. Streifendatei `band/band5.txt` mit einer leeren Zeile (Vorspann) und `00010101` (die Ziffer 5: Spuren p, 4, 1). Die Eingabeadresse 05 wird mit dem Adressteil 0000 zur Adresse 0005 verknüpft; der AC erhält den **Inhalt** der Zelle 0005, nicht die Zahl 5.

```
Trommeldatei:
#PROT STUFE=0
0000: 00004600030
0001: 00000000011
0002: 00000000020
0005: 00000000042
Tastenfolge:
H G R

# band/band5.txt:
(leere Zeile)
00010101
```

```
$ python3 c8205_interp.py --leser1 band/band5.txt Tests/leser-ziffer.txt ; echo "Rückkehrcode: $?"
=== Test leser-ziffer (#PROT STUFE=0) ===
Ende: Unbedingter Stop (US:=1) | Knoten: 63 | BZ=0003 AC=00000000042 AR=0000 | FE=0 FUE=0 BS=0 US=1 E=0 A=0
  V1=0 V2=0
Rückkehrcode: 4
```

Der AC enthält den Inhalt 42 der Zelle 0005. Ohne `--leser1` endet derselbe Test mit der Fehlermeldung aus Kapitel 7. Steht vor der Ziffer die Zeile `11111111` („Irrung Zeichen“, `band/band5i.txt`), ist das Ergebnis dasselbe: die Zeile wird übersprungen.

```
$ python3 c8205_interp.py --leser1 band/band5i.txt Tests/leser-ziffer.txt ; echo "Rückkehrcode: $?"
=== Test leser-ziffer (#PROT STUFE=0) ===
Ende: Unbedingter Stop (US:=1) | Knoten: 63 | BZ=0003 AC=00000000042 AR=0000 | FE=0 FUE=0 BS=0 US=1 E=0 A=0
  V1=0 V2=0
Rückkehrcode: 4
```

Mit dem Zeichen : (Zeile `00001101`) und dem Eingabe-Halt `0001: 00000000010` springt das Programm statt zu addieren nach 0015 (Sprung-Umstellung „ja“). Mit der Anwahl `4500 030` statt `4600 030` und `--leser2` liest derselbe Test von Leser 2:

```
$ python3 c8205_interp.py --leser2 band/band5.txt Tests/leser2-ziffer.txt ; echo "Rückkehrcode: $?"
=== Test leser2-ziffer (#PROT STUFE=0) ===
Ende: Unbedingter Stop (US:=1) | Knoten: 63 | BZ=0003 AC=00000000042 AR=0000 | FE=0 FUE=0 BS=0 US=1 E=0 A=0
  V1=0 V2=0
Rückkehrcode: 4
```

Stanzer: Zeichen 5 und Punkt

Programm: `0000: 00004600030` (K1 anwählen), `0001: 00000250030` (Ausgabe der Ziffer 5, Ausgabe-Adresse 025x), `0002: 00000730030` (Ausgabe des Punktes, 073x), `0003: 00000000020`.

```
$ python3 c8205_interp.py --stanzer1 out/band.txt Tests/stanzer-zeichen.txt ; cat out/band.txt
=== Test stanzer-zeichen (#PROT STUFE=0) ===
Ende: Unbedingter Stop (US:=1) | Knoten: 74 | BZ=0004 AC=00000000000 AR=0000 | FE=0 FUE=0 BS=0 US=1 E=0 A=0
  V1=0 V2=0
00010101
00111011
```

Die Datei `out/band.txt` enthält die Zeilen `00010101` und `00111011`. Der Ordner `out/` muss nicht existieren; er wird angelegt. Ohne `--stanzer1` endet der Test mit der roten Fehlermeldung aus Kapitel 7.

Stanzer-Datei gegen eine Soll-Datei prüfen

Soll der Test die gestanzte Datei selbst bewerten, steht die Dateiprüfung im Teil `Prüfungen:` (Abschnitt 3.4). Die Testdatei bringt Gerät und Prüfung mit; die Soll-Datei `soll.txt` liegt neben der Testdatei und enthält die Zeilen `00010101` und `00111011`:

```
Trommeldatei:
#PROT STUFE=0
--stanzer1 out/band.txt
0000: 00004600030
0001: 00000250030
0002: 00000730030
0003: 00000000020
Tastenfolge:
H G R
Prüfungen:
US=1
--pruefe-stanzer1 soll.txt
```

```
$ python3 c8205_interp.py Tests/stanzer-pruefe.txt ; echo "Rückkehrcode: $?"
=== Test stanzer-pruefe (#PROT STUFE=0) ===
Ende: Unbedingter Stop (US:=1) | Knoten: 74 | BZ=0004 AC=00000000000 AR=0000 | FE=0 FUE=0 BS=0 US=1 E=0 A=0
  V1=0 V2=0
Ergebnispruefung:
  US: soll=1 ist=1 OK
  Stanzer 1: soll=Tests/soll.txt ist=out/band.txt OK
Test wahr
Rückkehrcode: 0
```

Abweichender Inhalt, fehlende Soll-Datei oder fehlende Ist-Datei ergibt `Test falsch` (Rückkehrcode 6; Beispiel mit `diff` in Abschnitt 6.5). Für die Schreibwerk-Datei gilt dasselbe mit `--sw-ausgabe` und `--pruefe-sw-ausgabe`.

K0: Eingabe aus einer Datei

Programm: `0000: 00004700030` (K0 anwählen), `0001: 00000000011` (Eingabe-Halt mit Löschen), `0002: 00000000020`, dazu die Konstante `0007: 00000000007`. Die Datei `band/k0.txt` enthält eine Zeile `7`.

```
$ python3 c8205_interp.py --k0 datei=band/k0.txt Tests/k0-ziffer.txt ; echo "Rückkehrcode: $?"
=== Test k0-ziffer (#PROT STUFE=0) ===
Ende: Unbedingter Stop (US:=1) | Knoten: 63 | BZ=0003 AC=00000000007 AR=0000 | FE=0 FUE=0 BS=0 US=1 E=0 A=0
  V1=0 V2=0
Rückkehrcode: 4
```

Der AC enthält den Inhalt der Zelle 0007. Mit `SW=1` vor `G` (Schreibwerk statt Tastatur) ist auch das Zeichen B eingebbar; an der Pult-Tastatur nicht (Kapitel 7).

8.4 Diagnose, Speicherauszug und Prüfungen

Verkürzter Gruppen-Transport mit Rechtsverschiebung (Test `s1-R1`): Der AC startet mit 6; der Befehl `4001 542` überträgt ihn in die Spur 40xx, solange der AC nicht negativ wird. Der Leerbefehl 164 (Adresse 0020) löst die Diagnose mit `#SPEI-0020 4000-4037` aus.

```
Trommeldatei:
#DOK-0020 s1-R1: GX-Transport mit Rechtsverschiebung
#SPEI-0020 4000-4037
#PROT STUFE=0
0000: 00003000211
0001: 00004001542
0003: 00000000020
0010: 00000020164
3000: 00000000006
Tastenfolge:
H G R
```

Achtung Der Leerbefehl steht in Zelle **0010**, nicht in 0002. Nach einem X-Abbruch (verkürzte Gruppen-/Wiederholungsbefehle mit B = 5 oder 7) holt der Rechner den nächsten Befehl ohne Wartezeit aus dem Sektor, der gerade unter dem Kopf liegt: die Spur kommt aus dem Befehlszähler (hier Spur 00), der Sektor ist der Sektor der letzten bearbeiteten Zelle plus zwei Wortzeiten (hier 4016: Sektor 16 + 32 = 50, modulo 40 = 10, oktal). Der Befehl muss also **nicht** in Zelle b+1 stehen. Steht er dort, findet der Interpreter in 0010 den Wert 0, also den Sprung nach 0000, und das Programm läuft endlos. Danach läuft das Programm bei 0003 weiter.

```
$ python3 c8205_interp.py Tests/s1-R1.txt ; echo "Rückkehrcode: $?"
=== Test s1-R1 (#PROT STUFE=0) ===
-- Diagnose --
#DOK-0020 s1-R1: GX-Transport mit Rechtsverschiebung
#SPEI-0020 4000-4037
4000: 00000000000
4001: 00000000006
4002: 00000000000
… (4003 bis 4015: jeweils 00000000000) …
4016: 00000000003
4017: 00000000000
… (4020 bis 4037: jeweils 00000000000) …
Ende: Unbedingter Stop (US:=1) | Knoten: 95 | BZ=0004 AC=00000000001 AR=0000 | FE=0 FUE=0 BS=0 US=1 E=0 A=0
  V1=0 V2=0
Rückkehrcode: 4
```

Speicherzeilen gekürzt (die Adressen laufen oktal).

Es werden **zwei** Zellen geschrieben. In Zelle 4001 landet der AC (6); danach schiebt die Resultatbehandlung ihn nach rechts (AC = 3). Bei R = 1 gilt z_{32} als Vorzeichen: $z_{32} = 1$, der AC zählt also als „negativ“ (X-Bedingung). Operation und Resultatbehandlung werden noch mit einer weiteren Zelle ausgeführt (4016 := 3, AC danach 1), danach bricht der Befehl ab.

Test mit Prüfungen: Der folgende Test lädt eine Zahl, addiert eine zweite und prüft Register, Flags und zwei Speicherzellen (ein Beispiel mit `Test falsch` steht in Abschnitt 6.5):

```
Trommeldatei:
#DOK-0001 Test Ladebefehl und Addition
#PROT STUFE=0
0000: 00003000211 # AC := <3000>
0001: 00003001210 # AC := AC + <3001>
0002: 00000001164 # Leerbefehl, Diagnose #DOK-0001
0003: 00000000020 # Stopp
3000: 00000000100
3001: 00000000045
Tastenfolge:
H G R
Prüfungen:
BZ=0004 AC=00000000145 AR=0000 FE=0 FUE=0 BS=0 US=1 E=0 A=0 V1=0 V2=0
3000: 00000000100
3001: 00000000045
```

```
$ python3 c8205_interp.py Tests/pruef-wahr.txt ; echo "Rückkehrcode: $?"
=== Test pruef-wahr (#PROT STUFE=0) ===
-- Diagnose --
#DOK-0001 Test Ladebefehl und Addition
Ende: Unbedingter Stop (US:=1) | Knoten: 90 | BZ=0004 AC=00000000145 AR=0000 | FE=0 FUE=0 BS=0 US=1 E=0 A=0
  V1=0 V2=0
Ergebnispruefung:
  BZ: soll=0004 ist=0004 OK
  AC: soll=00000000145 ist=00000000145 OK
  AR: soll=0000 ist=0000 OK
  FE: soll=0 ist=0 OK
  FUE: soll=0 ist=0 OK
  BS: soll=0 ist=0 OK
  US: soll=1 ist=1 OK
  E: soll=0 ist=0 OK
  A: soll=0 ist=0 OK
  V1: soll=0 ist=0 OK
  V2: soll=0 ist=0 OK
  3000: soll=00000000100 ist=00000000100 OK
  3001: soll=00000000045 ist=00000000045 OK
Test wahr
Rückkehrcode: 0
```

Rückkehrcode 0 (`Test wahr`). Dieselbe Testdatei ohne `Prüfungen`: ergäbe 4.

9 Kurzreferenz

Aufruf

```
python3 c8205_interp.py [--mdverz V] [--out V|F.log] [--leser1 P] [--leser2 P] [--stanzer1 P] [--sw-ausgabe P]
   [--k0 datei=P]
       [--knotentest] [--NO-TERMOUT] [--pruefe-stanzer1 D] [--pruefe-sw-ausgabe D] TEST.txt ...
./c8205_interpreter.sh Tests/x.txt [SCHALTER]        # Protokoll nach Test-Protokolle/x.log
```

Schalter	Wirkung (Standard)
`--out V` / `--out F.log`	Ordner (Protokoll in V/<Test>/protokoll.txt) / eine Datei (nur 1 Test); Standard Test-Protokolle/
`--leser1 P` `--leser2 P`	Streifen (Datei/Ordner) für K1 / K2 – Zeilen aus 0/1: w v u p 8 4 2 1 · auch als Zeile der Testdatei
`--stanzer1 P`	Ausgabedatei Stanzer (K1), eine 0/1-Zeile je Zeichen · auch als Zeile der Testdatei
`--sw-ausgabe P`	Schreibwerk-Ausgabe (K2) in P/sw-ausgabe.txt statt ans Terminal · auch als Zeile der Testdatei
`--k0 datei=P`	K0-Eingabe aus Datei/Ordner (ein Zeichen/Name je Zeile); ohne Schalter vom Terminal · auch als Zeile der Testdatei
`--pruefe-stanzer1 D` `--pruefe-sw-ausgabe D`	Ist-Datei gegen Soll-Datei D vergleichen (setzt --stanzer1 / --sw-ausgabe voraus; nur 1 Test) · auch als Zeile in Prüfungen:
`--knotentest`	Knotenzeilen Kxxx … ins Protokoll (aus)
`--NO-TERMOUT`	Terminal-Ausgaben des Interpreters unterdrücken (case-sensitiv); aus
`--mdverz V`	Verzeichnis der Regelwerk-Dateien (Knotengraph, Variablentabelle T2); Standard markdown-dateien/

Testmarke	Wirkung
`#DOK-xxxx Text`	Text bei Befehl 164 mit Adresse xxxx
`#SPEI-xxxx s-e`	Speicher s…e bei 164/xxxx (auch ohne #DOK)
`#SPUR-xxxx a`	ganze Spur von a bei 164/xxxx
`#PROT STUFE=n`	0 kein · 1 Bildschirm (Standard) · 2 Datei

Token	Wirkung
`H R`	Halt / Rechnen (R: H:=0, US:=0, Start)
`G`	Generallöschen (nur bei H); danach US:=1; K0 angewählt; SW vorher setzen
`M+ M- M`	M halten / loslassen / antippen (OR v 025)
`E`	Einzelschritt; unter M: BZ+1
`Ü · ST`	AR:=7777, ORv240 (nur bei H) · ORv240, US:=0 (nur bei US/BS)
`SW=1 SW=0`	Schreibwerk / Tastatur (T)
`BS=1/0 SP=1/0`	einrastende Schalter
`K0 … K3`	Kanaltaste (nach G); Anwahlbefehl im Programm übersteuert sie

Anwahl	Kanal	Eingabe	Ausgabe
`4700 030`	K0	Tastatur (T) / Schreibwerk (SW)	– (Ausgabe-Halt = Stub)
`4600 030`	K1	Leser 1	Stanzer 1
`4500 030`	K2	Leser 2	Schreibwerk (Terminal, Dunkelgelb)
`4400 030`	K3	frei	frei (nicht benutzt, Stub)

Testdatei	Aufbau
`Trommeldatei:`	Zeilen `aaaa: wwwwwwwwww [# …]` (4 + 11 Oktalstellen), Testmarken, Geräte-Zeilen
`Tastenfolge:`	Token; Ende mit R oder im Halt
`Prüfungen:` (optional)	1 Zeile `BZ=… AC=… FE=… …`, Zeilen `aaaa: w…`, `--pruefe-…`-Zeilen

Endzeile	Lesart
`Ende: Grund | Knoten: n | BZ AC AR | FE FUE BS US E A V1 V2`	Zahlen oktal; E=1/A=1: Gerät wartet; Knoten fassungsabhängig
Farben	Dunkelgelb = Ausgabe des Rechners · Rot = Meldung, beginnt mit FEHLER:
Rückkehrcode	0 wahr · 4 ohne Prüfung · 6 falsch · 8 Laufzeitfehler · 16 Vorab-Fehler · 2 ungültiger Aufruf

Anhang A: Glossar

Begriff	Erklärung
Akkumulator (AC)	Rechenregister (33 Bit); einziger Weg zum Abspeichern in die Trommel
Befehlszähler (BZ)	Register (12 Bit) mit der Adresse des nächsten Befehls
Adressregister (AR)	Register (12 Bit), wählt die Trommelzelle für jeden Zugriff
Operationsregister (OR)	Register (9 Bit) mit dem Operationsteil des aktuellen Befehls (BAZ)
Befehlsregister (BR)	Adreßteil (12 Bit) und OR (9 Bit); im Protokoll `BR=aaaa ooo`
Flag	Speicher für 0 oder 1 (E, A, T, SW, K1–K3, FE, FÜ, BS, US, V1, V2, H, SP); eine Lampe zeigt ihn, ein Schalter kann ihn setzen
Trommel	Magnettrommelspeicher: 4096 Zellen à 33 Bit
Sektor / Bahn (Spur)	32 Sektoren je Bahn; 128 Bahnen. Eine Adresse besteht aus Bahn (7 Bit) und Sektor (5 Bit)
Oktal	Zahlen zur Basis 8 (Ziffern 0–7); alle Zahlen in Testdateien
Knoten, Konnektor	Elemente des Knotengraphen (Anweisung, Test, Phasenzuordnung, Konnektor); der Interpreter läuft ihn ab
Phase I bis IV	die vier Abschnitte eines Befehlszyklus: Zähleraufstellung, Befehlsentnahme, Adressaufstellung, Ausführung
ANWEISUNG-nnn	Beschreibung eines Knotens in Tabelle T2; im Interpreter eine eigene Funktion
Stub	Meldung „an dieser Stelle fehlt eine Umsetzung“ bzw. „außerhalb des definierten Betriebs“; der Test hält an
Testmarke	Zeile mit #, die nur der Testdurchführung dient (`#DOK`, `#SPEI`, `#SPUR`, `#PROT`)
Zeichenlage KB/GB	Klein-/Großbuchstaben-Lage; beim Schreibwerk zufällig gesetzt, umgeschaltet durch Umschaltzeichen
Kanal K0 bis K3	Anwahl der Ein-/Ausgabegeräte (Tabelle in Kapitel 5)
T / SW	Subflags für K0: Pult-Tastatur bzw. Schreibwerk statt Tastatur (alternativ)
Sprung-Umstellung	Eigenschaft eines Zeichens der Kodetabelle: der Eingabebefehl wird in einen Sprung umgestellt (z. B. `:`, `.`)
Irrung Zeichen	Lochbild 11111111 (alle acht Spuren); am Leser von der Paritätsprüfung ausgenommen und übersprungen
Tastenfolge	die Bedienhandlungen am Pult, die den Ausgangszustand festlegen
Trommeldatei	Speicherinhalt (Ausschnitt), mit dem ein Test beginnt
Dateiprüfung	Vergleich der vom Lauf erzeugten Stanzer- bzw. Schreibwerk-Datei mit einer Soll-Datei (`--pruefe-…`)
Rückkehrcode	Zahl, die der Interpreter dem Betriebssystem liefert (0, 2, 4, 6, 8, 16; Abschnitt 4.4)
Wortzeit	Zeit, in der ein Wort unter dem Kopf durchläuft (104 µs); im Interpreter 0

Anhang B: Grenzen des Interpreters

- **Zeiten sind 0:** Es gibt keine Wartezeiten und keine Zeitmessung; die Pufferung der Ausgabe (1-Zeichen-Puffer) wird nicht nachgebildet.
- Kanal K3 wird nicht benutzt (Festlegung des Autors); Programme auf K3 und ein Ausgabe-Halt ohne Ausgabegerät liegen außerhalb des definierten Betriebs und enden mit einem Stub.
- Die Schreibwerk-Ausgabe geht ohne `--sw-ausgabe` an das Terminal; mit dem Schalter in die Datei `sw-ausgabe.txt`. Ein Warten auf den Schalter SW findet nicht statt. Die Stanzer-Ausgabe entspricht dem Regelwerk (eine 0/1-Zeile je Zeichen); den Dateinamen bestimmen Sie mit `--stanzer1`. Der Stanzer hat keine Terminal-Ausgabe.
- Die früheren Schalter `--k0 tastatur` und `--leser-ende` gibt es nicht mehr: ohne `--k0` wird vom Terminal gelesen, Leser und `--k0 datei=` warten am Ende still auf eine neue Datei.
- Ein Transportloch außerhalb des Vorspanns (nach dem Original der Paritätsprüfung unterworfen) ist im Streifenformat nicht darstellbar und nicht umgesetzt; die Auslegung steht noch aus.

- Der Rückkehrcode bezieht sich auf genau einen Test, bei mehreren Testdateien auf den zuletzt bearbeiteten.
- Die Taste `M` (ohne `E`) startet bei einem Sprung mit Rückkehrabsicht in BR (025, 027, 125, 127) in dieser Fassung bei Knoten 19; die Sprungoperation läuft über Knoten 40. Der in der Quelle (Abschnitt 7) beschriebene Start bei Knoten 54 ist im Interpreter **nicht umgesetzt**.
- Nach einem X-Abbruch (B = 5 oder 7) steht der Folgebefehl nicht in Zelle b+1, sondern im Sektor, der zeitlich fällig ist (Abschnitt 8.4); ältere Tests mit Folgebefehl in b+1 laufen sonst in eine Endlosschleife.
- B und D (Pult-Tasten) haben keinen Code. WM = Tabulator (Eingabe-Adresse 075), IB = Eingabe-Adresse 036; die Tabellenzeile 52 („Transport“) wird nicht benutzt.
- Der Interpreter ist wortlautgetreu nach dem Knotengraphen: ANWEISUNG-001 gilt in Phase I und III; die Resultatbehandlung gilt für jeden Befehl außer den Leer-Befehlen 164/166; bei B = 5 und 7 entscheidet bei R ≠ 1 das Bit z_0, bei R = 1 das Bit z_{32} über den X-Abbruch.

Anhang C: Verweise auf das Regelwerk

Thema	Dokument
Rechner, Phasen, Register, Trommel	PDF „Rechnerbeschreibung“ (Quellen: Überblick_und_Gesamtaufbau, Datenstruktur…, Magnettrommel)
Tasten, Schalter, Kanäle, Leser, Stanzer, Schreibwerk	PDF „Bedienpult und Peripherie“ (Quellen: Bedienpult_mit_Schaltern, Lochstreifenleser)
Befehle und Formeln	PDF „Einzelbefehlsliste“ (Quelle: Befehlsliste.md)
Zeichenkodes, Tabelle 1	PDF „Kodetabellen“
Knotengraph und Variablentabelle T2	PDF „Knotengraph“; Quellen Eingänge_der_Knoten.md, Beschreibung_der_Knoten.md
Testdefinition, Testdatei und Protokoll	PDF „Testhandbuch“ (Quelle: Durchführung_eines_Tests.md)
Cache der BAZ-Wirkungen	baz-wirkungstabelle.md (vom Python-Interpreter nicht gelesen)